

DOCUMENTATION TECHNIQUE – Projet F1



Table des matières

1. Présentation générale du projet

2. Architecture générale du projet

2.1 Architecture logicielle

2.2 Description des couches

3. Modèle de données (Base MySQL)

3.1 Tables principales

3.1 Tables secondaire

4. Fonctionnalités principales

4.1 Interface publique

4.2 Interface user

4.3 Sécurité

5. API et services externes

5.1 API jolpy

5.2 Django

6. Déploiement

6.1 Prérequis

6.2 Étapes de déploiement

7. Sécurité et bonnes pratiques

8. Tests

9. Outils utilisés

10. MCD

11. MLD

1. Présentation générale du projet

Nom du projet : F1 – Amélie Bonhomme

Technologies principales :

- Backend : Django + Django REST Framework
- Frontend : Vue.js 3
- Base de données : MySQL
- API externe : Ergast (via Jolpy)
- HTML / CSS / JavaScript
- Axios (requêtes HTTP)
- GitHub

Objectif du projet

Développer une application web permettant :

- d'afficher les équipes de Formule 1,
- de présenter les circuits en europe,
- de permettre aux utilisateurs de créer un compte,
- de gérer leurs équipes favorites,
- de gérer leurs circuits préférés,
- de laisser des commentaires,
- d'avoir une interface moderne, responsive et dynamique.

2. Architecture générale

2.1 Architecture logicielle

Le projet suit une architecture SPA + API REST :

Code

/back (Django)

 /app

 /models.py

 /views.py

 /serializers.py

 /urls.py

/frontend (Vue.js)

 /src

 /components

 /views

 /router

 /composables

Séparation des rôles

- **Django** : API REST, logique métier, sécurité, validation, accès MySQL.
- **Vue.js** : Interface utilisateur, navigation, appels API, gestion user

2.2 Description des couches

Backend Django

- **Models** : représentation des objets (User, Équipe, Circuit, Favoris...)
- **Serializers** : validation + conversion Python ↔ JSON
- **Views (APIView / ViewSet)** : login, register, favoris, préférés, commentaires
- **URLs** : définition des endpoints API
- **ORM Django** : communication avec MySQL

Frontend Vue.js

- **Views** : pages principales (Équipes, Circuits, Profil...)
- **Components** : éléments réutilisables (cards, modals...)
- **Vue Router** : navigation SPA
- **Composables Vue (useAuth)**
- **Axios** : communication avec l'API Django

3. Modèle de données (Base MySQL)

3.1 Tables principales

Table f1_user :

Champ	Type	Description
id_user	VARCHAR(50)	Identifiant unique
prenom	VARCHAR(50)	Prénom
nom	VARCHAR(50)	Nom
sexe	VARCHAR(50)	Sexe
mail	VARCHAR(50)	Email
date_naissance	DATE	Date de naissance
mdp	VARCHAR(50)	Mot de passe hashé
login	VARCHAR(50)	Identifiant de connexion

Table equipe :

Champ	Type	Description
id_equipe	VARCHAR(50)	Identifiant
nom	VARCHAR(50)	Nom de l'équipe
pilote_principal	VARCHAR(255)	Pilote principal
image_pilote	TEXT	Image pilote principal
pilote_second	TEXT	Pilote secondaire
image_pilote2	VARCHAR(50)	Image pilote secondaire
logo	TEXT	Logo de l'équipe
description	TEXT	Information sur l'équipe

Table circuit :

Champ	Type	Description
id_circuit	VARCHAR(50)	Identifiant
nom_circuit	VARCHAR(255)	Nom du circuit
lien_site	TEXT	Lien officiel
image_circuit	TEXT	Image
description_circuit	TEXT	Description

Table commentaire :

Champ	Type	Description
id_commentaire	VARCHAR(50)	Identifiant
texte	TEXT	Contenu
date_	DATE	Date
id_user	VARCHAR(50)	FK → f1_user

3.2 Tables secondaires

Table favoris :

Fait le liens entre les écuries et les utilisateurs

Champ	Type	Description
id_user	VARCHAR(50)	FK → f1_user
id_equipe	VARCHAR(50)	FK → equipe
id	INT	Identifiant interne

Table prefere :

Fait le lien entre les circuits et les utilisateurs.

Champ	Type	Description
id_user	VARCHAR(50)	FK → f1_user
id_circuit	VARCHAR(50)	FK → circuit
id	INT	Identifiant interne

4. Fonctionnalités principales

4.1 Interface publique

Donne la possibilité à l'utilisateur de se connecter.

4.2 Interface utilisateurs

Accessible uniquement via authentification.

- Liste des équipes F1
- Détails d'une équipe
- Liste des circuits
- Détails d'un circuit
- Affichage des commentaires
- Design responsive
- Ajouter / retirer une équipe en favori
- Ajouter / retirer un circuit préféré
- Poster un commentaire

4.3 Sécurité

- Hashage des mots de passe (make_password / check_password)
- Validation stricte des données :
 - login unique
 - mot de passe minimum
 - email valide
 - sexe valide
 - commentaire non vide
 - favoris / préférés non dupliqués
- Vérification de l'existence des entités (user, équipe, circuit)

5. API et services externes

5.1 API Jolpy

Utilisée pour :

- récupérer des données externes F1 (si applicable),
- enrichir les informations affichées.

Fonctionnement :

1. Requête HTTP depuis Vue.js
2. Réception des données JSON
3. Intégration dans la page classement

5.2 Django REST API

Endpoints principaux :

- /api/users/
- /api/equipes/
- /api/circuits/
- /api/favoris/
- /api/prefere/
- /api/commentaires/

Format : **JSON**

6. Déploiement

6.1 Pré-requis

- Python 3.10+
- Node.js 18+
- MySQL
- Serveur web (Apache)

6.2 Étapes de déploiement

1. Installation des dépendances Django
2. Installation des dépendances Vue.js
3. Configuration du `.env` Django
4. Configuration du `.env` Vue.js
5. Migration de la base MySQL
6. Build du frontend Vue.js
7. Déploiement sur serveur

7. Sécurité et bonnes pratiques

- Hashage des mots de passe
- Validation backend systématique
- Vérification des entités (user, équipe, circuit)
- Protection contre les doublons
- Aucune confiance dans le frontend
- Séparation stricte frontend / backend
- API REST sécurisée

8. Tests

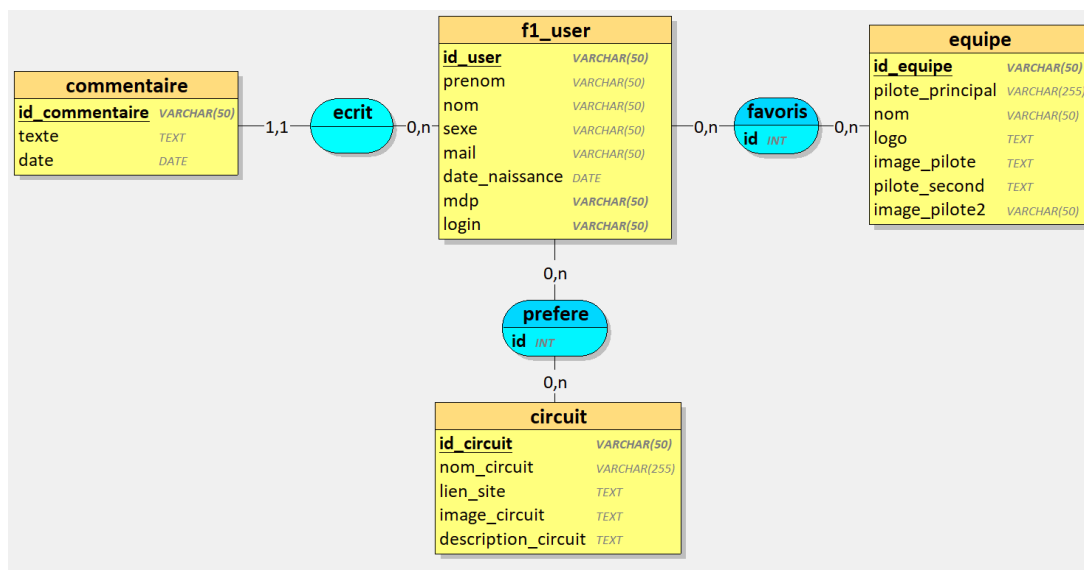
Tests manuels réalisés :

- Tests CRUD Django
- Tests API REST
- Tests Vue.js (affichage, navigation)
- Tests d'ajout / suppression favoris
- Tests d'authentification
- Tests responsive

9. Outils utilisés

- Django
- Django REST Framework
- Vue.js
- Axios
- MySQL
- Git / GitHub
- VS Code

10. MCD



11. MLD

